



Mansoura University
Faculty of Computers and Information
Department of Computer Science



Spring 2026

[SWE143] Software Project Management

Lecture 01: Introduction

Level 300: software engineering program

Muhammad Haggag Zayyan, Ph.D

Course Syllabus

- Introduction
 - Software Development Projects and Stakeholders
 - Software Development Activities
 - Project Initiation
 - Project Goals
 - Work Breakdown Structure
 - Estimating
 - Project Scheduling
 - Project Closing
-

Topics Description (1)

Introduction to the Course	Introduction to the course: motivations, goals, and a brief history of project management
Software Development Projects and Stakeholders	What is a project and how it differs from other types of work. Projects and their environment: internal and external influences. The project stakeholders. Distinguishing characteristics of software development projects. Types of software development projects.
(Traditional) Software Development Activities	Goals and organization of the (technical) activities which are part of any software development. (That is, the building blocks of a software development project.)
Project Initiation: Feasibility and Project Authorization	Qualitative and quantitative techniques to assess a project proposal. Choosing what projects are worth and what should not be started. Techniques for evaluating merits and demerits of alternative implementation plans. Make or buy. Writing a feasibility study.
Formalizing the Project Goals	Formalizing the project goals: MoSCoW and SMART objectives. Writing a project scope document. The project scope document and software requirements.

Topics Description (2)

Deciding the work to be performed (Work Breakdown Structure)	Moving from goals to activities: the Work Breakdown Structure (WBS). Types of WBS. WBS decomposition and building styles. The WBS dictionary.
Estimating	Moving from the WBS to a scheduled plan. Effort, duration, and manpower and the relationship among these three values. Uncertainty in planning. Estimating duration, effort, and manpower. Program Evaluation and Review Technique (PERT) Expert judgement and reasoning by analogy. Algorithmic estimation techniques (high level introduction).
Function Points	Overview of the "Function Points" technique. Using FP to estimate software development projects. Benefits and limitations.
COCOMO	Overview of the "Constructive Cost Modeling" technique. Using COCOMO to estimate software development projects. Benefits and limitations.
Project Scheduling	Moving from a WBS to a scheduled plan: identifying and managing dependencies. Critical Path method. Resource leveling.
Project Closing	Closing a project and ensuring a proper transition to new activities.

Course resources

- Primary reference:
 - ❑ Introduction to Software Project Management, by Adolfo Villafiorita. CRC Press, 2014
- Secondary reading:
 - ❑ An Introduction to Project Management, Fifth edition, by Kathy Schwalbe, Schwalbe Publishing, 2015
- Lecture notes

Motivations (1/3)

- Many courses in computer science and electronic engineering focus on the **technical** aspects and on the **notations** to develop good software (e.g., how to do testing; how to write requirements; how to model systems; UML; Java)
- However, in order to build good software, a well defined and managed **process**, which organizes the activities in an efficient and controlled way
- Technical Skills: These include programming, modeling, testing, and requirement specification. They teach you how to perform tasks within software development.
- Process Skills: These define when, how, and in what order those technical tasks should occur. They help manage complexity, coordination, and quality.

Motivations (2/3)

- Consider the following:
 - Writing good requirements is no good if you don't have a controlled process, for instance, to **accept changes** and **trace revisions**
 - The techniques to do testing are useless if you **don't have any time left** to do testing (because, for instance, you underestimated the development time and are late with a release)
 - The development of a software system requires to execute, monitor, and control **various activities which have little or nothing to do with writing code**. Consider, for instance: training users, packaging a product, managing publicity and communication, writing user documentation

Motivations (3/3)

- If you want to deliver on time and within budget a product which has the quality properties agreed upon (be it a software or any other product), you need:
 - A process to define a schedule, a budget, and agree on the (quality) characteristics of a product
 - A list of techniques to define, agree, plan, execute, and monitor: goals, quality, time, and costs

Skills and some goals of the course

- Managing a (software development) project, thus, requires specific competences, skills, and techniques
- Some of the questions you will be able to answer at the end of this course include:
 - How do I **estimate** how long it will take to complete a task?
 - Break down tasks into smaller components and estimate the time for each.
 - How much am I going to **charge** for a project?
 - Project management is typically charged 10-15% of the total cost, based on the project *type*, size, and *team*.
 - How do I keep the team **motivated** and ensure projects are fulfilling and an occasion to learn, grow, and advance in one's career?
 - Foster a positive work environment by rewarding, staying connected, and encouraging creativity.

Skills and some goals of the course

- How do I deal with project **risks**?
 - Common risks include code issues, aggressive deadlines, unmet expectations, low productivity, and budget concerns.
- How do I **assess** whether the project is on time, on budget, on schedule?
 - Use visual charts with real-time data updates to spot changes and make updating a routine.
- How do I control the **quality** of the final output?
 - Build a quality culture through standardization, focus on the end user, and test early and often in the SDLC, even if it means slowing down the process.

Software Project Management

The Project Management techniques are intrinsically multidisciplinary ...

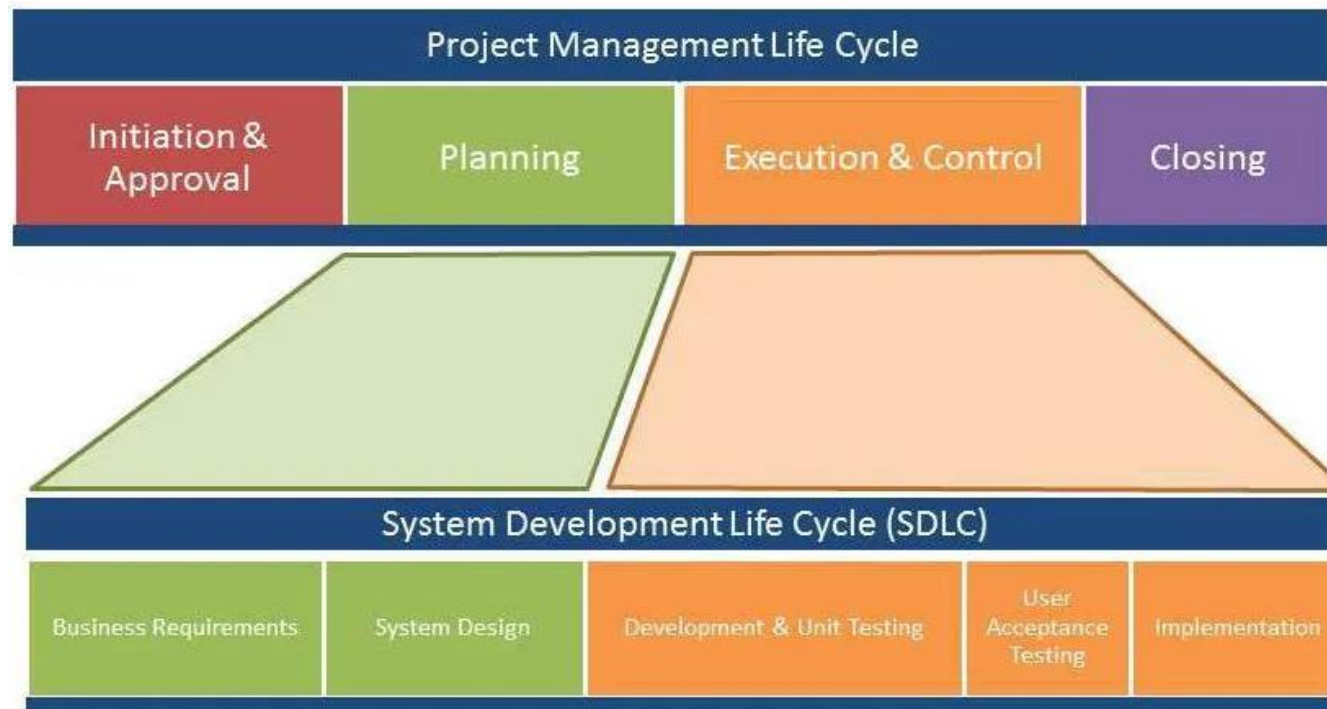
... what you will learn in this course is applicable to virtually any other (engineering) domain.

There are however certain characteristics that make the management of software projects particularly interesting.

Software Project Management

- Software project management is interesting and **challenging** because:
 - The product is **intangible**
 - The product is **uniquely flexible** (e.g. different sizes; different constraints)
 - Many software projects are 'one-off'
 - This means they are unique, custom-built solutions tailored to meet the specific needs of a client or organization, rather than being mass-produced or reused across multiple clients.
 - The development process is **uniquely flexible**
 - **Size and complexity are increasing** exponentially
 - Human lives might depend on software running as expected (consider the control system of an airplane)- **safety critical systems**

The SDLC vs. Project Management



- The project management life cycle (PMLC) adds two phases that the Software Development Life Cycle (SDLC) does not have:
 - **Initiation and Approval** – before the SDLC even starts
 - **Closing** – after the SDLC completes (however the SDLC doesn't actually complete – Maintenance Phase)
 - SDLC has one additional phase:
 - **Maintenance** → occurs **after deployment**, not formally part of the PMLC.
 - The project management life cycle encompasses all the activities of the project, while the systems development life cycle focuses on realizing the product requirements.
-

Phases of software project management



Terms

- **Project Initiation:** Starts when an organization recognizes a need for a new or improved capability and explores how to address it.
 - **Project Charter:** A concise document that outlines project objectives, scope, and responsibilities to gain approval from key stakeholders.
 - **KPIs** (Key Performance Indicators): Measurable values that demonstrate how effectively a project or business is achieving its objectives.
 - **Forecasts:** Predictions about future project outcomes based on current data, trends, and assumptions.
 - **Post Mortem:** تحليل ما حدث بعد انتهاء المشروع A review conducted after a project is completed to assess its successes, challenges, and lessons learned.
 - **Project Punchlist:** الملاحظات A list of remaining tasks or issues that need to be addressed before a project is considered complete.
 - **Reporting:** The process of regularly communicating project progress, status, and key metrics to stakeholders.
-

Example:

Software Project Management (SPM) Phases for
Launching a New Website

1. Project Initiation

- Create a project charter and business case. Gather initial stakeholder input to decide if the project is viable.
 - **Deliverable:** Project Initiation Document (PID) that outlines the project's goals, business need, stakeholders, and high-level requirements.
 - **Example Deliverable:** A document that states, "Our goal is to build an e-commerce website for XYZ Retail by Quarter 4, aimed at increasing online sales by 20%."
-

2. Project Planning

- Develop a detailed project plan that includes timeline, resources, budget, scope, and risk management strategies.
 - **Deliverables:**
 - **Scope Statement:** Clearly defines the website's features (e.g., user registration, payment gateway).
 - **Work Breakdown Structure (WBS):** Breaks the project down into smaller tasks, such as design, coding, content creation, and testing.
 - **Risk Management Plan:** Identifies risks like delays in content creation or issues with payment integration.
 - **SMART Goals:** "Complete website design by the end of month 2."
-

3. Project Execution

- Implement tasks such as developing the user interface, coding, and integrating the backend. Conduct status meetings to track progress.
 - **Deliverables:**
 - **Kick-off meeting:** Align the team on project expectations.
 - **Team Development:** Assign roles like web developers, designers, and content writers.
 - **Updates and Progress Reports:** Regular status reports that detail work completed (e.g., "Homepage design completed, awaiting feedback").
 - Example: Developers begin coding the e-commerce functionality while designers work on website visuals.
-

4. Project Performance and Monitoring

- Use KPIs to track progress, such as checking if development is on schedule, if budget is being maintained, and if the site is meeting quality standards.
 - **Deliverables:**
 - **Performance Tracking:** Regular reports to monitor progress against the timeline and budget.
 - **Issue Resolution:** Address any delays or problems that arise, such as a delay in coding due to unanticipated technical issues.
 - Example: A report shows that development is lagging, so the team adjusts the timeline and adds extra resources.
-

5. Project Close

- Complete all tasks, perform a final project review, and deliver the website to the client. Hold a post-mortem meeting to evaluate what went well and what could be improved.
 - **Deliverables:**
 - **Final Budget and Report:** Provide a final account of all expenses and report on whether goals were met.
 - **Project Punchlist:** Resolve any remaining tasks (e.g., fixing minor bugs).
 - **Final Deliverable:** Fully functional website launched to the public.
 - Example: The website is now live, but a minor bug is found that needs a quick fix, which is noted for future updates.
-

A Brief History of (Software) Project Management

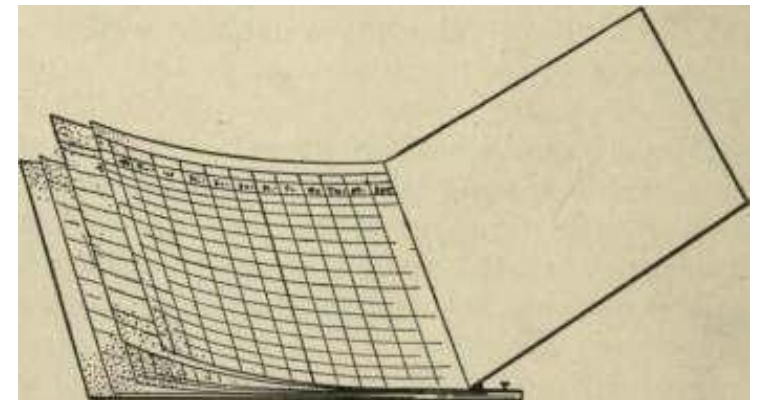
Frederick Winslow Taylor



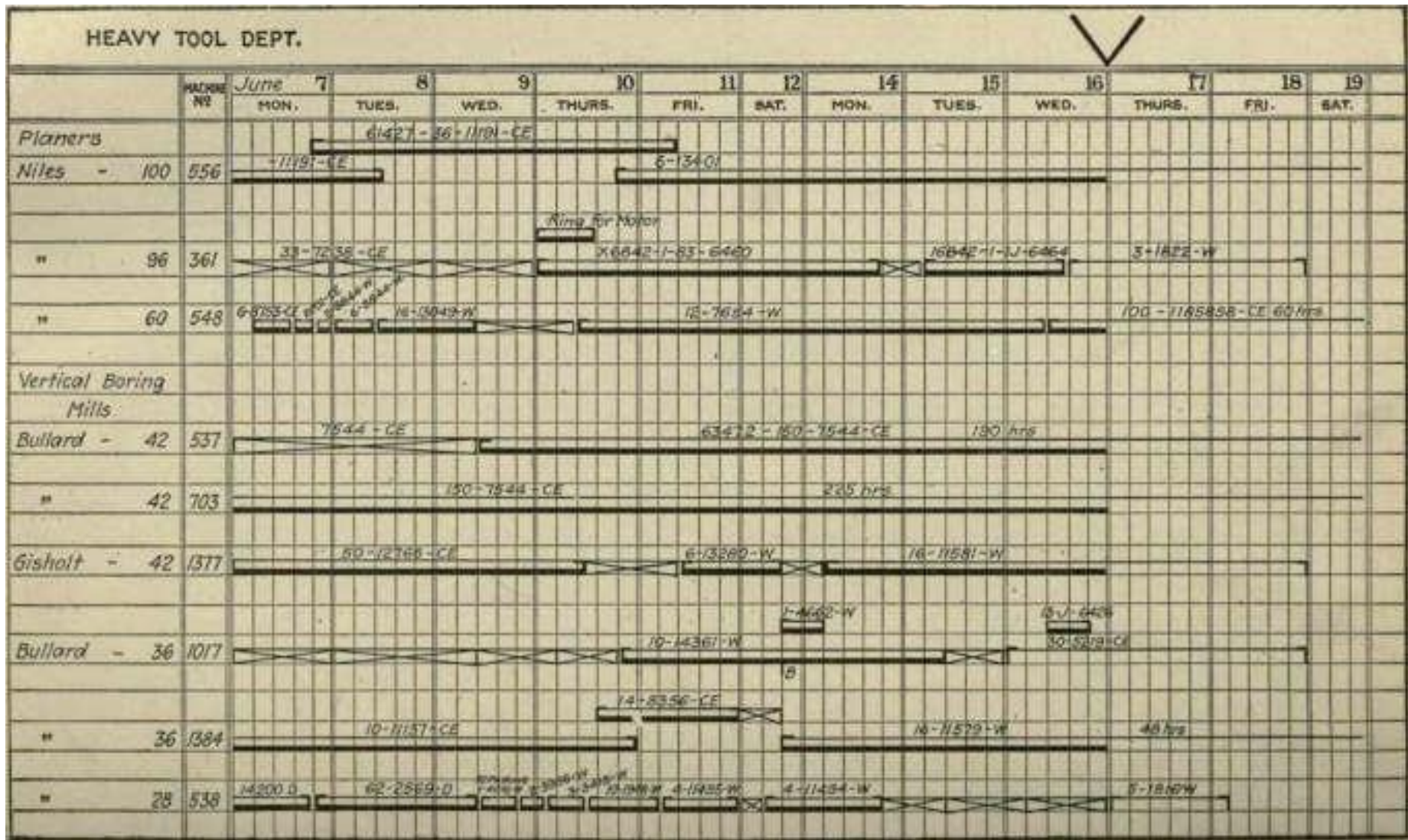
- Taylor's "Scientific Management" is the first theory of work and management
- Beginning of 1900
- A negative view of workers (they perform at the slowest rate which goes unpunished)
- Some more interesting characteristics:
 - Scientific definition of work
 - Scientific selection of personnel
 - Sharing of responsibilities between workers and management
 - Incentives and rest periods (to make workers more efficient)

Henry Gantt

- Gantt's "Gantt chart" notation is still used today to schedule projects
- Defined during the First World War
- First used to schedule and monitor work and progress in ship building: distinction between work and progress
- His book available for download from archive.org



Example of Gantt Chart



A Gantt chart is a type of bar chart that illustrates a project schedule. This chart lists the **tasks** to be performed on the **vertical axis**, and time intervals on the horizontal axis. The **width** of the **horizontal bars** in the graph shows the **duration** of each activity.

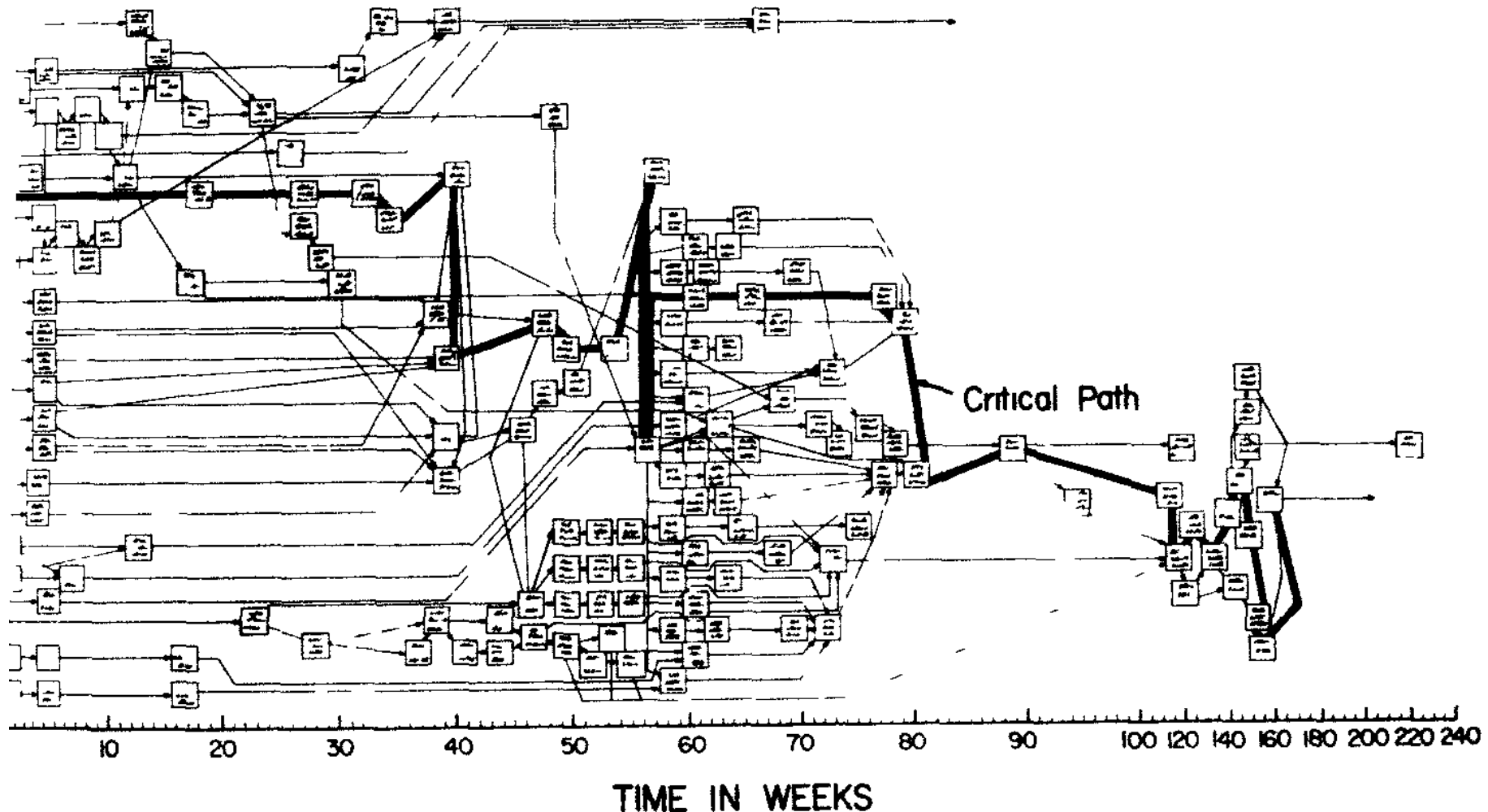
Fifties and Sixties

- 2nd World War:
 - Manhattan Project: process flow diagrams
- 1957 **CPM** (Critical Path Method)
 - Mathematically based algorithm for scheduling a set of project activities, used to plan maintenance activities in plants
 - Dupont + Remington Rand UNIVAC team
 - No fundamental changes to date
- 1958 **PERT** (Program Evaluation and Review Technique)
 - U.S. Navy Polaris missile program (Booz Allen & Hamilton (management consulting firm) working as ORSA team for Lockheed Missile System)



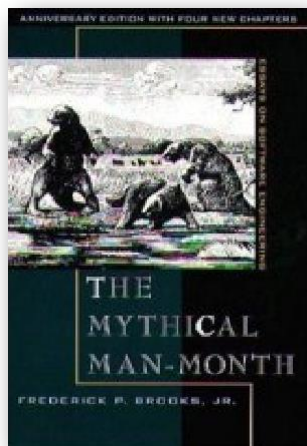
Motivation for CPM and PERT

What activities could delay the project delivery of the following plan?



Fifties and Sixties

- **1960's** Big Government contracts (Vietnam, nuclear power plants, NASA Apollo): standardization and automation
 - PERT/COST and WBS become compulsory in Government's sponsored projects
 - Earned Value Analysis (EVA) is defined
 - Configuration management
 - Project organizations (PMI, IPMA) promote profession and techniques
 - (1961) IBM uses PM commercially
- **1970** Software development gets into the equation
 - EVA developed for monitoring schedule and cost
 - Waterfall model for software development
 - The mythical man-month highlights many pitfalls of software development



The Computer Revolution

- **1980's**
 - Hardware and software proliferation make PM tools accessible to smaller firms
 - Hardware capacity grows exponentially (Moore's Law) and so does software
 - Estimation models (FP and COCOMO) are introduced to predict software complexity
- **1990's**
 - Total quality
 - Leaner, quicker, more responsive organizations
- **Today**
 - Web application and new application distribution models
 - Development with components and frameworks
 - Agility, quick interaction, constant feedback

<https://tinyurl.com/SW-PM-2026>

